

# Operating Systems

A Complete Course Presentation

Units 1 - 5



**SAGE**  
UNIVERSITY  
—BHO—

# UNIT 1

Introduction to Operating Systems

Architecture, Goals & Structure, Basic Functions, System Calls, OS Types



**SAGE**  
UNIVERSITY  
—BHOPL—

# What is an Operating System?

An Operating System (OS) is system software that acts as an intermediary between the user/application programs and the computer hardware

Manages hardware resources — CPU, memory, storage, and I/O devices — on behalf of running programs

Provides a convenient and consistent environment for users to execute programs

Controls and coordinates the use of hardware among various application programs

Examples: Windows, Linux, Unix, macOS, Android, iOS



# Two Views of an Operating System

## User View

OS as a convenient interface between the user and the hardware

Focus is on ease of use and convenience

Resource utilization is generally not a user priority

Interfaces: Command Line Interface (CLI), Graphical User Interface (GUI)

## System View

OS as a Resource Manager — allocates CPU, memory, I/O devices and files among competing requests

OS as a Control Program — manages execution of user programs to prevent errors and misuse

Ensures efficient, fair and secure use of resources

Focus is on orderly and efficient system operation



# Goals of an Operating System

Convenience — make the computer system easier and more convenient to use

Efficiency — allow computer system resources to be used in an efficient manner

Ability to Evolve — permit effective development, testing and introduction of new functions without interrupting service

Throughput — maximize the amount of work accomplished in a given time

Reliability — ensure the system performs its functions correctly and consistently

Protection & Security — safeguard data and resources from unauthorized access



# Basic Functions of an Operating System

Process Management — creating, scheduling and terminating processes

Memory Management — allocating and deallocating memory space as needed

File Management — organizing, storing, retrieving, naming and protecting files

Device / I-O Management — controlling and coordinating I/O devices through device drivers

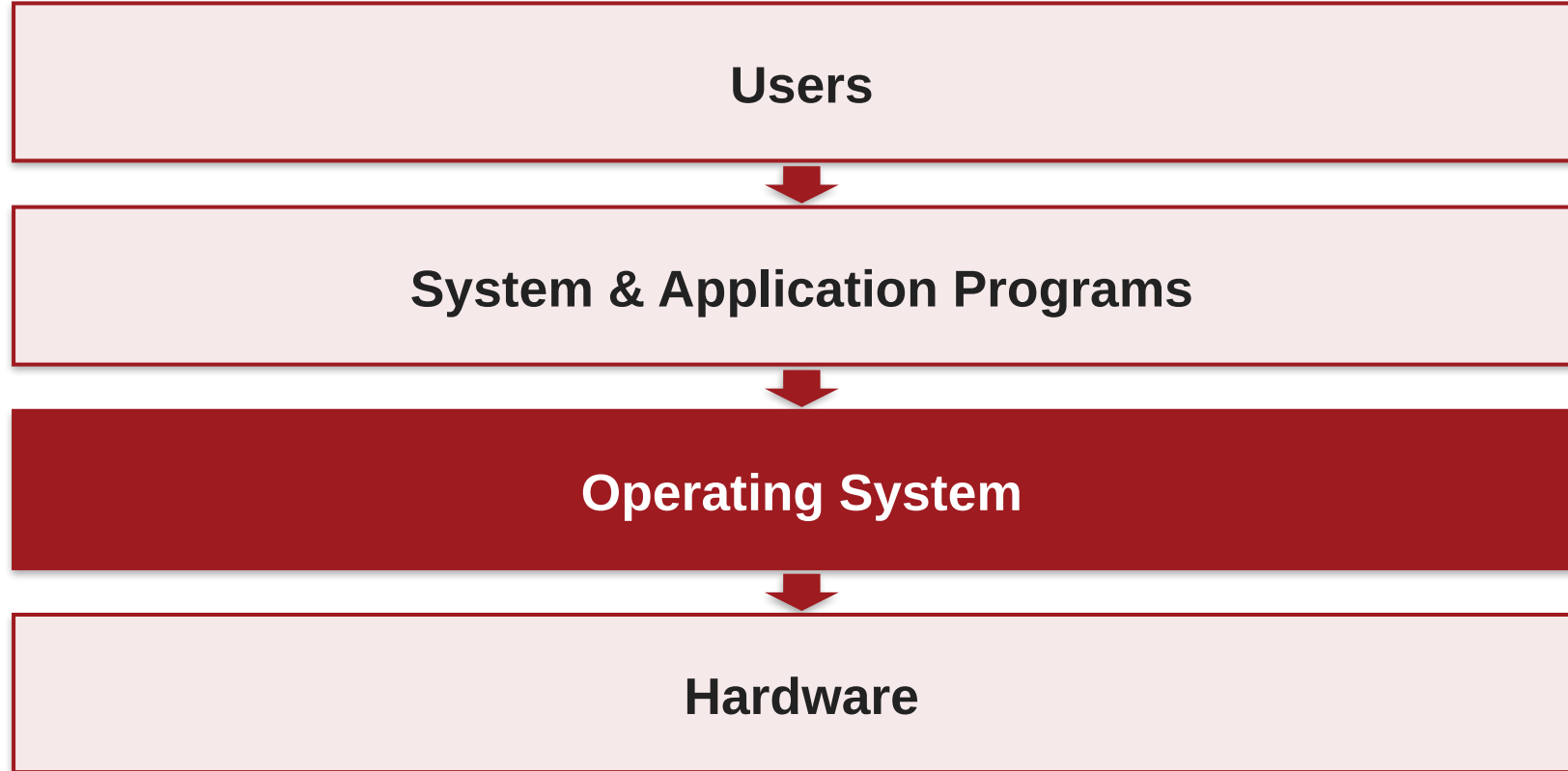
Security & Protection — controlling access to system resources

User Interface — providing CLI/GUI for user interaction

Networking — enabling communication between systems over a network



# Computer System Architecture



*Each layer interacts only with the layer directly below it, forming an abstraction hierarchy from hardware up to the end user.*



# OS and Hardware Interaction

The OS communicates with hardware through device drivers and device controllers

Dual-Mode Operation: User Mode (restricted) and Kernel Mode (privileged) protect hardware from user program errors

A mode bit in hardware indicates the current mode; privileged instructions execute only in kernel mode

Interrupts signal the CPU when a hardware device needs attention (I/O completion, errors)

Direct Memory Access (DMA) lets devices transfer data to/from memory without CPU intervention, improving efficiency

The interrupt vector holds addresses of interrupt service routines for fast handling



# OS Structure: Simple / Monolithic Systems

Entire OS runs as a single large program in kernel mode

All functions — process, memory, file and device management — are tightly interconnected

Example: Original MS-DOS, early UNIX systems

## **Advantages:**

Good performance due to direct function calls; simple to design

## **Disadvantages:**

Difficult to maintain and extend; a bug in one part can crash the whole system



# OS Structure: Layered Approach

OS is divided into a number of layers (levels), each built on top of lower layers

Bottom layer (layer 0) is the hardware; the highest layer is the user interface

Each layer uses only the functions and services of the layer directly below it

Advantage: simplifies design, debugging and verification — each layer tested independently

Disadvantage: careful layer planning needed; layer definition is difficult; adds call overhead

Example: THE operating system



# OS Structure: Microkernel Architecture

Moves as much functionality as possible out of the kernel into user space

Kernel provides only minimal core services: IPC, basic scheduling, basic memory management

File systems, device drivers and other services run as user-level processes

Communication between components happens via message passing

Advantages: easier to extend, more reliable and secure, easier to port

Disadvantages: performance overhead from increased inter-process communication

Examples: Mach, QNX



# OS Structure: Modular & Hybrid Systems

Modular (Loadable Kernel Modules): kernel has a core set of components and dynamically links in additional services as needed

Similar to the layered approach but more flexible — any module can call any other module

Hybrid systems combine features of monolithic and microkernel designs for both performance and modularity

Examples: Linux (loadable modules), Windows and macOS (hybrid kernels)



# Comparison of OS Structures

## Advantages

Monolithic: fast performance, simple internal communication

Layered: easy to design, debug and modify

Microkernel: highly reliable, secure and portable

Modular: flexible, efficient, dynamic loading

## Limitations

Monolithic: hard to maintain, poor fault isolation

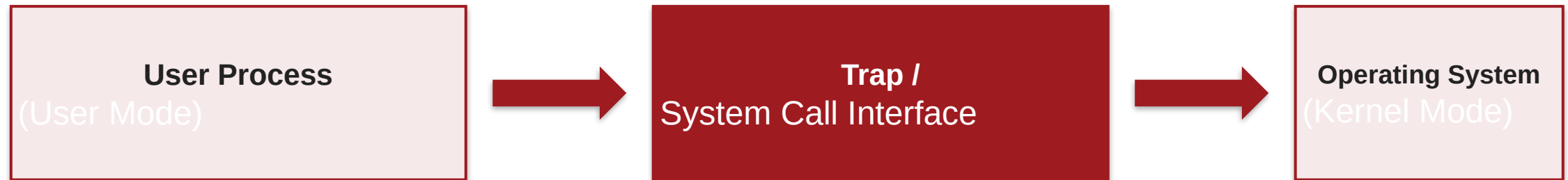
Layered: layer definition is difficult, added overhead

Microkernel: slower due to message-passing overhead

Modular: complex module-interface management



# System Calls: Concept & Mechanism



*A system call is the interface through which a running program requests a service from the OS kernel. A trap instruction switches the CPU from user mode to kernel mode; control returns to the user process once the OS completes the request.*

# Types of System Calls

Process Control — create, terminate, load, execute, wait, abort a process (fork, exec, wait, exit)

File Management — create, delete, open, close, read, write a file

Device Management — request/release a device, read, write, reposition

Information Maintenance — get/set time, date, system data, process attributes

Communication — create/delete a connection, send/receive messages (pipe, shared memory)

Protection — get/set file permissions, allow or deny access



# Batch Processing Systems

Jobs with similar needs are grouped (batched) together and executed as a group without user interaction

An operator or automatic job sequencer submits jobs; output is collected later

No direct interaction between the user and the executing job

Reduces CPU idle time compared to manual/serial processing

Disadvantages: difficult to debug, long turnaround time, jobs must wait in queue



# Multiprogramming

Keeps multiple jobs (programs) in main memory simultaneously

CPU switches to another job whenever the current job needs to wait, e.g. for I/O

Maximizes CPU utilization by ensuring the CPU always has a job ready to execute

OS decides which job in memory to execute next through job scheduling

Foundation concept for time-sharing and multitasking systems



# Multitasking and Time-Sharing Systems

Multitasking allows a single CPU to execute multiple tasks by rapidly switching between them

Time-sharing is a multitasking variant that lets many users share the system simultaneously via time slices

Each user gets a small quantum of CPU time in round-robin fashion, giving the illusion of simultaneous execution

Provides fast response time and interactive use of the computer

Requires more sophisticated CPU scheduling and memory management than batch systems



# Parallel and Distributed Systems

## Parallel Systems

Also called multiprocessor systems — two or more CPUs in close communication, sharing memory/bus

Symmetric Multiprocessing (SMP): each processor runs an identical copy of the OS

Asymmetric Multiprocessing: one master processor controls the system, others are worker processors

Advantages: increased throughput, economy of scale, increased reliability

## Distributed Systems

A collection of physically separate, independent computers connected via a network

Each processor has its own local memory; processors communicate via communication lines

Advantages: resource sharing, computation speed-up, reliability, communication

Types: Client-Server systems, Peer-to-Peer systems



# Real-Time Operating Systems (RTOS)

Designed for applications with strict time constraints on process execution

Hard Real-Time Systems: critical tasks must complete within a guaranteed time — missing a deadline causes system failure (e.g. robotics, aircraft control)

Soft Real-Time Systems: critical tasks get priority, but deadlines are not strictly guaranteed (e.g. multimedia, video conferencing)

Requires predictable, deterministic scheduling behavior

Examples: VxWorks, QNX, RTLinux



# Unit 1 Summary

Operating System: an intermediary between the user, applications and hardware

Goals: convenience, efficiency, and ability to evolve

Structures: Monolithic, Layered, Microkernel, Modular / Hybrid

System calls provide the interface between processes and the OS kernel

OS types by execution style: Batch, Multiprogrammed, Multitasking / Time-Shared, Parallel, Distributed, Real-Time

